```
    a[0] = "Hello"
    a[1] = "World"
    fmt.Println(a[0], a[1])   //prints the o/p : Hello World
    fmt.Println(a)         //prints the o/p : [Hello World]

    primes := [10]int{2, 3, 5, 7, 11, 13}
                // short hand declaration to create array
    fmt.Println(primes)     // prints the o/p : [2 3 5 7 11
13 0 0 0 0], remaining                              elements
filled with 0
```

The Go programming language allows multi-dimensional arrays. The following example prints a 2D array:

```
func main() {
     var twod [2][3]int
    for i := 0; i < 2; i++ {
        for j := 0; j < 3; j++ {
            twod[i][j] = i + j
        }
    }
    fmt.Println("Elements : ", twod)
}
```

Slices

A slice is a convenient, flexible and powerful wrapper on top of an array. Slices do not own any data on their own. They are just the references to existing arrays.

The type *[]T* is a slice with elements of type *T*. A slice is formed by specifying two indices, a low and high bound, separated by a colon:

```
func main() {
names := [4]string{"John","Paul","Ann","Manu",}
fmt.Println(names)
a := names[0:2]
b := names[1:3]
fmt.Println(a, b)
O/p
[John Paul Ann Manu]
[John Paul] [Paul Ann]
```

Maps

Go provides another important data type named the *map*, which maps unique keys to values. The declaration is as follows:

```
/* declare a variable, by default map will be nil*/
Syntax : var map_variable map[key_data_type]value_data_type
 Eg: var namedepartment map[string]string

You must use make function to create a map.
Syntax : map_variable = make(map[key_data_type]value_data_
type)
Eg : namedepartment= make(map[string]string)

package main
import "fmt"
func main() {
    personSalary := make(map[string]int)
    personSalary["Liz"] = 12000
    personSalary["Amy"] = 15000
    personSalary["Mike"] = 9000
    fmt.Println("personSalary map contents:", personSalary)
}
Prints the o/p:
personSalary map contents: map[Liz:12000 Amy:15000
Mike:9000]
```

Pointers in Go

Go supports pointers, allowing you to pass references to values and records within your program.

The syntax for pointer declaration is as follows:

```
var pointer_name *pointer_data_type
Eg:    var b *int

func main() {
    b := 255
    a := &b
    fmt.Println("address of b is", a)
    fmt.Println("value of b is", *a)
    *a++
    fmt.Println("new value of b is", b)

}
```

It prints the following output:

- The address of b is 0x10414020.
- The value of b is 255.
- The new value of b is 256.

Structure in Go

To define a structure, you must use the *type* and *struct* statements. The *struct* statement defines a new data type, with multiple members for your program. The *type* statement binds a name with the type which is 'struct' in our case. The format of the *struct* statement is as follows:

```
type struct_variable_type struct {
   member definition;
   member definition;
...
   member definition;
}
```

Once a structure type is defined, it can be used to declare variables of that type using the following syntax: